

Parcial Final

Diego Fernando Florez Mayo

Lenguaje de Programación III
Ingeniería de Sistemas(5 Semestre)

Guacarí, Valle del Cauca

9 de octubre

INFORME TÉCNICO: INTEGRACIÓN Y VALIDACIÓN DE UN SERVICIO DE SEGUIMIENTO LOGÍSTICO

Introducción

El presente proyecto consiste en el desarrollo de un prototipo de seguimiento logístico que permite consultar el estado de los envíos mediante un número de guía, almacenar la información obtenida en una base de datos y registrar los resultados de las solicitudes realizadas al sistema.

La solución se desarrolló utilizando Python y Flask para crear el servicio web, una API simulada para proporcionar los datos de los envíos y SQLite para almacenar y consultar la información. Además, se incorporó un sistema de registros para observar el comportamiento de la aplicación y se realizaron pruebas de carga con Locust para evaluar su respuesta ante múltiples solicitudes.

Es importante aclarar que la API utilizada es simulada y que SQLite se empleó como base de datos de demostración. Por tanto, los resultados corresponden a un prototipo local y no a una integración comprobada con una empresa logística real ni con una base de datos institucional.

Objetivo general

Desarrollar y validar un prototipo de seguimiento logístico que integre una API de consulta de envíos, almacenamiento de información, registro de eventos y pruebas de carga para evaluar su funcionamiento.

Tecnologías utilizadas

- **Python:** lenguaje utilizado para desarrollar la lógica del sistema.
- **Flask:** framework empleado para crear los servicios web.
- **Requests:** biblioteca utilizada para realizar solicitudes HTTP a la API simulada.
- **SQLite:** motor de base de datos utilizado para almacenar y consultar los envíos.
- **Logging:** herramienta de Python utilizada para registrar eventos, advertencias y tiempos de respuesta.
- **Locust:** herramienta utilizada para ejecutar pruebas de carga.
- **Visual Studio Code y terminal de Windows:** herramientas utilizadas durante el desarrollo y las pruebas.

Arquitectura de la solución

El sistema se organizó en componentes independientes para facilitar su comprensión y mantenimiento.

1. **Aplicación principal (app.py):** recibe las solicitudes de consulta, coordina la comunicación con la API y la base de datos, y devuelve las respuestas al usuario.
2. **Servicio de integración (api_service.py):** realiza las solicitudes HTTP a la API simulada y controla respuestas de error, como las solicitudes que superan el tiempo de espera o las guías inexistentes.
3. **Base de datos (database.py):** crea la tabla de envíos y permite guardar y consultar la información mediante SQLite.
4. **API simulada (api_simulada/api.py):** proporciona información de prueba para diferentes números de guía.
5. **Registros (logs/app.log):** almacena eventos relacionados con las consultas, errores y tiempos de respuesta.
6. **Pruebas de carga (tests/locustfile.py):** define los escenarios utilizados para comprobar el comportamiento del servicio bajo solicitudes concurrentes.

El flujo comienza cuando se consulta un número de guía en la aplicación principal. Esta solicita la información a la API simulada. Si la respuesta es correcta, los datos se almacenan en SQLite y se devuelve el resultado. Si la guía no existe o se presenta un problema de comunicación, el sistema devuelve una respuesta de error y registra el evento correspondiente

Integración y validación de la API

Para simular un servicio externo de seguimiento se desarrolló una API con Flask, disponible localmente en el puerto 5001. Esta API devuelve información de prueba en formato JSON.

Se definieron tres envíos de ejemplo:

- **LG-1001:** estado «En tránsito», origen Cali y destino Cartagena.
- **LG-1002:** estado «Entregado», origen Bogotá y destino Medellín.
- **LG-1003:** estado «Preparando envío», origen Barranquilla y destino Cali.

La aplicación principal consulta esta API mediante solicitudes HTTP. También contempla errores de comunicación y tiempos de espera.

Se realizaron dos pruebas principales:

- **Guía existente (LG-1001):** el sistema recibió una respuesta HTTP 200, obtuvo los datos del envío y los almacenó en la base de datos.
- **Guía inexistente (LG-9999):** el sistema recibió una respuesta HTTP 404 y devolvió un mensaje indicando que no se encontró el envío.

Estas pruebas permitieron comprobar el manejo de respuestas correctas y de solicitudes para guías inexistentes. Sin embargo, al tratarse de una API simulada, no se verificó la comunicación con un proveedor logístico externo real.

Almacenamiento y consulta de datos

Se utilizó SQLite para implementar una base de datos local denominada envios.db. En ella se creó una tabla llamada envios, con campos para identificar el registro, almacenar el número de guía, el estado, el origen, el destino, la transportadora y la fecha de la última actualización.

Cuando se consulta una guía existente mediante el servicio principal, la información obtenida de la API se almacena en la base de datos. Posteriormente, el servicio de consulta local permite recuperar el registro sin solicitar nuevamente los datos a la API simulada.

Las pruebas realizadas fueron las siguientes:

- Consultar la guía LG-1001 mediante el servicio principal y verificar su almacenamiento.
- Consultar la guía LG-1001 directamente en la base de datos y comprobar que se recupera la información.
- Consultar la guía LG-9999 en la base de datos y verificar que se devuelve una respuesta HTTP 404 cuando el registro no existe.

Limitación: SQLite se utilizó como alternativa de demostración. No se estableció una conexión con una base de datos institucional, por lo que esa integración queda pendiente de disponer de los accesos, las credenciales y la configuración autorizada correspondientes.

Observabilidad y registro de eventos

Para observar el funcionamiento de la aplicación se implementó un sistema de registros mediante el módulo logging de Python. Los eventos se guardan en el archivo logs/app.log.

El sistema registra el inicio de las consultas, las respuestas correctas de la API, el almacenamiento de los envíos, los errores de consulta y los tiempos de respuesta medidos en segundos.

Por ejemplo, durante la consulta de la guía LG-1001, los registros mostraron que la API respondió correctamente, que el envío se almacenó y que se midió el tiempo de respuesta. En la consulta de LG-9999, se registró una advertencia porque la guía no existía.

También se implementaron las rutas / y /salud para comprobar que la aplicación responde y que el servicio se encuentra activo.

Estos mecanismos facilitan la identificación de errores y permiten revisar el comportamiento del prototipo durante las pruebas. Los registros obtenidos corresponden a las ejecuciones realizadas en el entorno local.

Pruebas de carga con Locust

Se utilizó Locust para evaluar el comportamiento del servicio bajo múltiples solicitudes HTTP. Se configuró una prueba con 10 usuarios simulados, una tasa de incorporación de 2 usuarios por segundo y una duración aproximada de 62 segundos.

El escenario incluyó consultas a una guía existente (LG-1001) y a una guía inexistente (LG-9999). En el archivo de pruebas se configuró `catch_response=True` para considerar como válidas las respuestas esperadas: HTTP 200 para la guía existente y HTTP 404 para la guía inexistente.

Resultados de la prueba corregida

Indicador	Resultado
Duración	62 segundos
Solicitudes totales	400
Solicitudes para guía existente	294
Solicitudes para guía inexistente	106
Tiempo medio de respuesta	26,68 ms
Percentil 95 (P95)	40 ms
Percentil 99 (P99)	46 ms
Tiempo máximo registrado	72 ms
Solicitudes por segundo (RPS)	6,37
Fallos inesperados contabilizados por Locust	0

La prueba registró 400 solicitudes y no contabilizó fallos inesperados. Las respuestas HTTP 404 de las guías inexistentes se clasificaron como resultados válidos porque forman parte del escenario de prueba y representan el comportamiento esperado del sistema.

Estos resultados muestran que el prototipo respondió a las solicitudes realizadas durante esta ejecución. No obstante, no constituyen una garantía de rendimiento en producción, ya que las pruebas se realizaron localmente, con una API simulada y una base de datos SQLite.

Limitaciones identificadas

Durante el desarrollo se identificaron las siguientes limitaciones:

- Los datos de los envíos son simulados y no proceden de una empresa de transporte real.
- La base de datos utilizada es SQLite local, no una base de datos institucional.
- Las pruebas de carga se realizaron en un entorno local y con una cantidad limitada de usuarios simulados.
- No se evaluó el comportamiento del sistema ante grandes volúmenes de tráfico ni ante las condiciones de una infraestructura de producción.
- La disponibilidad y exactitud de la información dependen de los datos de prueba configurados en la API simulada.

Como trabajo futuro, se propone integrar un proveedor logístico real cuando se disponga de acceso autorizado, configurar la conexión institucional correspondiente y ampliar las pruebas con diferentes cantidades de usuarios y escenarios de error.

Conclusiones

El desarrollo permitió construir un prototipo funcional para consultar el estado de envíos, almacenar información en una base de datos local y recuperar los registros guardados. También se implementó el manejo de respuestas HTTP, el registro de eventos y la medición de tiempos de respuesta.

Las pruebas funcionales confirmaron el comportamiento esperado para una guía existente y una guía inexistente. Por su parte, la prueba de carga realizada con Locust registró 400 solicitudes durante aproximadamente 62 segundos, con un tiempo medio de respuesta de 26,68 milisegundos y sin fallos inesperados contabilizados.

En conclusión, el prototipo cumple los objetivos de demostración e integración planteados para este desarrollo. Sin embargo, la conexión con un proveedor logístico real y con una base de datos institucional no fue implementada, por lo que ambas se consideran aspectos pendientes para una futura versión.

Uso de herramientas de inteligencia artificial

Durante el desarrollo del proyecto de seguimiento logístico se utilizó ChatGPT como herramienta de apoyo para la planificación, programación, resolución de dudas y elaboración de la documentación técnica. Sin embargo, la inteligencia artificial no sustituyó mi participación en el desarrollo, ya que fui responsable de trabajar con el código, organizar los archivos, ejecutar la aplicación, comprobar los resultados y realizar las correcciones necesarias para lograr el funcionamiento conjunto de los componentes.

Actividades en las que recibí apoyo de ChatGPT

ChatGPT me proporcionó orientación para estructurar el proyecto de una manera más organizada, separando las funcionalidades en distintos archivos de Python. A partir de esta orientación, se planteó una arquitectura compuesta por una aplicación principal (app.py), un módulo de comunicación con la API (api_service.py), un módulo de acceso a datos

(database.py), una API simulada (api_simulada/api.py) y un archivo de pruebas de carga (tests/locustfile.py).

También recibí explicaciones y ejemplos sobre la utilización de Flask para crear las rutas del servicio web, Requests para realizar solicitudes HTTP, SQLite para almacenar la información de los envíos y Logging para registrar eventos y tiempos de respuesta. Adicionalmente, ChatGPT me orientó en la preparación de escenarios de prueba para comprobar las respuestas del sistema y en el uso de Locust para evaluar el comportamiento de la aplicación ante múltiples solicitudes.

En el componente documental, ChatGPT me ayudó a mejorar la gramática, la ortografía, la organización de los párrafos y la claridad de las explicaciones. También me proporcionó una estructura para presentar la introducción, los objetivos, la arquitectura, las tecnologías utilizadas, las pruebas realizadas, los resultados, las limitaciones y las conclusiones del proyecto.

Actividades desarrolladas por el estudiante

Mi participación fue fundamental durante el proceso de construcción, integración y validación del prototipo. Las principales actividades que realicé fueron las siguientes:

1. Organización del entorno de desarrollo. Preparé y utilicé el entorno de trabajo en Windows, trabajé con Python y organicé los archivos y carpetas necesarios para mantener separado el código de la aplicación, la API simulada, la base de datos, los registros y las pruebas.

2. Implementación y adaptación del código. Trabajé con los diferentes archivos de Python que componen la solución, incorporando y adaptando la lógica necesaria para que los módulos pudieran comunicarse entre sí. También revisé las propuestas de código recibidas y realicé ajustes cuando fue necesario para adecuarlas a la estructura real del proyecto.

3. Desarrollo de la API simulada. Participé en la preparación y comprobación del servicio que proporciona los datos de prueba de los envíos. Verifiqué el uso de números de guía de ejemplo y la respuesta del sistema cuando se consulta una guía existente o una que no se encuentra registrada.

4. Integración de los componentes. Comprobé el funcionamiento conjunto de la API simulada y la aplicación principal, verificando que las solicitudes HTTP permitieran obtener los datos esperados y que las respuestas pudieran ser procesadas por el sistema.

5. Implementación y validación del almacenamiento. Trabajé con la base de datos SQLite y comprobé que la información obtenida mediante la consulta de un envío pudiera almacenarse y recuperarse posteriormente. También verifiqué el comportamiento del sistema cuando se intenta consultar un registro que no existe en la base de datos.

6. Verificación de las rutas de la aplicación. Probé las rutas principales del servicio, incluyendo la ruta inicial, la comprobación de salud, la consulta de envíos y la recuperación de los datos almacenados. Estas pruebas me permitieron comprobar que las respuestas fueran coherentes con el comportamiento esperado.

7. Revisión y corrección de errores. Durante el desarrollo identifiqué situaciones que requerían ajustes en el código o en la configuración de las pruebas. Utilicé los mensajes de error, las respuestas HTTP y los resultados obtenidos durante la ejecución para revisar el funcionamiento de los componentes y corregir las inconsistencias detectadas. Algunos ejemplos concretos del proceso de validación fueron comprobar la respuesta HTTP 404 para una guía inexistente y ajustar la clasificación de las respuestas en Locust para distinguir los resultados esperados de los fallos inesperados.

8. Implementación y comprobación de la observabilidad. Revisé los registros generados por la aplicación en logs/app.log, comprobando que permitieran identificar las consultas, las respuestas de la API, el almacenamiento de la información y los tiempos de respuesta. También examiné los mensajes asociados a las consultas que no podían completarse correctamente.

9. Ejecución de pruebas de carga. Configuré y ejecuté las pruebas con Locust, utilizando 10 usuarios simulados y una tasa de incorporación de 2 usuarios por segundo. Posteriormente, revisé el reporte obtenido y trabajé en el ajuste de las pruebas para que las respuestas HTTP 200 y los HTTP 404 esperados se clasificaran correctamente de acuerdo con el escenario definido.

10. Análisis de los resultados. Revisé las estadísticas de la prueba de carga, incluyendo el número total de solicitudes, el tiempo medio de respuesta, los percentiles de latencia, el tiempo máximo y la cantidad de fallos inesperados contabilizados. Este análisis permitió documentar los resultados de manera más precisa y reconocer que corresponden a un entorno local de demostración.

11. Elaboración y organización de la documentación. Preparé y revisé el contenido del informe técnico y del archivo README.md, organizando la descripción del proyecto, las instrucciones de ejecución, las tecnologías utilizadas, las rutas disponibles, los resultados de las pruebas y las limitaciones identificadas.

12. Preparación de la entrega. Organicé los elementos necesarios para presentar el proyecto, incluyendo la estructura de carpetas, el archivo de dependencias, las instrucciones de

ejecución y las evidencias de funcionamiento. También revisé la importancia de excluir archivos locales innecesarios, como el entorno virtual, y de evitar la publicación de información sensible.

Correcciones realizadas durante la colaboración

La colaboración con ChatGPT también implicó un proceso de revisión y ajuste, ya que los ejemplos de código generados por la herramienta no se consideraron automáticamente correctos ni definitivos. Fue necesario contrastarlos con el funcionamiento real del proyecto, ejecutar las aplicaciones y revisar las respuestas obtenidas.

Uno de los ejemplos más claros fue el ajuste de las pruebas de carga. Inicialmente, Locust contabilizaba las respuestas HTTP 404 de las guías inexistentes como fallos, aunque esas respuestas eran esperadas para el escenario de prueba. Con la orientación recibida, revisé la configuración del archivo `tests/locustfile.py` y se incorporó el manejo de respuestas mediante `catch_response=True`, de manera que una respuesta HTTP 200 para una guía existente y una respuesta HTTP 404 para una guía inexistente pudieran clasificarse como resultados válidos cuando correspondiera.

Asimismo, revisé los resultados obtenidos en las pruebas funcionales, el almacenamiento de los envíos y los registros de ejecución para detectar posibles inconsistencias. Este proceso permitió comprender que generar código no es suficiente: también es necesario ejecutarlo, probarlo, interpretar los resultados y corregirlo cuando no cumple el comportamiento esperado.

La revisión documental siguió un proceso similar. ChatGPT proporcionó sugerencias para mejorar la redacción y la organización del informe, mientras que yo revisé el contenido y trabajé en su adaptación al proyecto desarrollado y a las evidencias disponibles.

Responsabilidad y aprendizaje

La experiencia me permitió reforzar conocimientos relacionados con Python, servicios web, solicitudes HTTP, bases de datos, manejo de errores, registro de eventos y pruebas de carga. Además, comprendí mejor la importancia de separar las responsabilidades de una aplicación en módulos independientes y de verificar cada componente antes de considerar terminado un proyecto.

El uso de inteligencia artificial se entendió como una forma de obtener orientación técnica, resolver dudas y acelerar determinadas tareas, manteniendo la necesidad de revisar y validar sus resultados. La responsabilidad sobre la integración, las pruebas, corrección, creación, la interpretación de los resultados y la presentación final del trabajo permaneció en el estudiante.

Finalmente, se reconoce que la solución desarrollada es un prototipo funcional basado en una API simulada y una base de datos SQLite local. No se estableció una conexión con un proveedor logístico real ni con una base de datos institucional. Estas limitaciones se documentaron para diferenciar los resultados comprobados de las funcionalidades que podrían implementarse en una versión futura.

En conclusión, ChatGPT fue una herramienta de apoyo para la planificación, la explicación de conceptos y la mejora de la documentación. Mi participación se centró en trabajar con la implementación, integrar los componentes, ejecutar las pruebas, revisar los resultados, corregir las inconsistencias detectadas, crear en base al proyecto y organizar la entrega del proyecto.